CCCS 104 - Data Structures and Algorithms

LEARNING TASK (LINEAR DATA STRUCTURE - QUEUE)

NAME/S: LANUZO, JESSICA T.
COLICO, CARL RENZ C.
TERCERO, MICHELLE S.

SECTION: BSCS 2-A

RATIONALE

*A Queue is a linear data structure that operates on a First-In-First-Out (FIFO) principle. This means that the first element added to the queue will be the first one to be removed. The main operations associated with a queue are enqueue (**to add an item**) and dequeue (**to remove an item**). In Queue, enqueue can only be performed to the rear or end of the queue, on the other hand dequeue can also be performed from the front of the queue that has the lowest value of elements.*

(a.) Queue Underflow:
Queue underflow happens when a dequeue operation is attempted on an empty queue.

(b.) Queue Overflow:
Queue overflow occurs when an attempt is made to add an element to a queue that has reached its maximum capacity.

A queue manages elements in a FIFO (First-in-First-out) order, with specific operations to add, remove, and access elements. It can be implemented using arrays or linked lists.

Such concepts of Queue LDS can be seen in real-life situations:

- *Booth ticket counters*
- *Customer Service (Online)*
- *BFS Graphs*
- *Order Processing*

Such applications of Queue LDS can also be seen in widely used various scenarios:

- *Task Scheduling: Queue manages the tasks or to be processed in a system of schedule thus making them to processed in the order they want it to received.*

- *E-Customer Service: Queues handle customer requests or orders in systems like call centers, where calls or service requests are managed in the order they are received.*
- *Online Shop (Order Processing): Shopping carts or checkout processes often use queues to handle and process customer orders in sequence.*
- *Resource Management (Memory management): Queues help manage memory allocation and deallocation in systems where tasks or processes are scheduled and managed dynamically.*

These various application Queues are used in real time due to their FIFO nature . Queues are versatile and essential in managing data and tasks in an orderly manner, making them crucial in various computational and real-world applications.

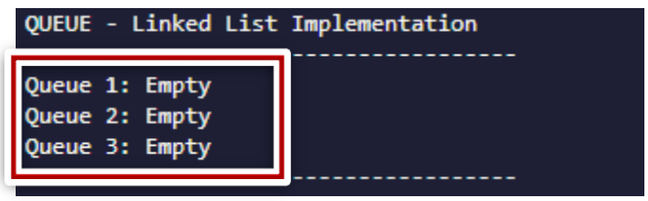
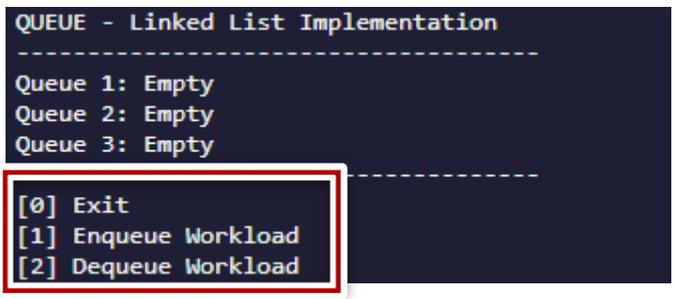
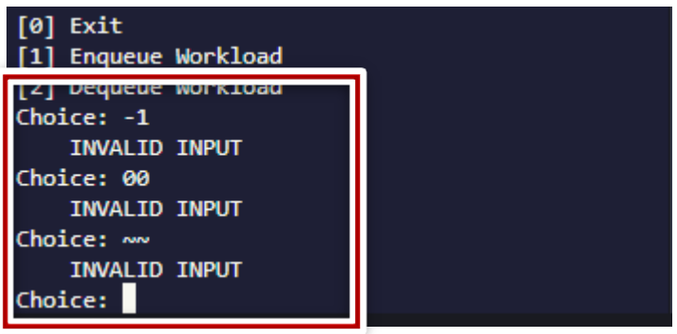
*For this Learning task, We then developed and demonstrates how **queues operate** using a **linked list structure**. This comes with balancing workloads among multiple queues, and managing queue operations in an interactive manner. That is design using a Python program. This learning task involves three queues that accept workloads consisting of integer elements. The system operates by enqueuing incoming workloads into the queue with the lowest total value, and dequeuing the element with the least value from all queues. It is essential to display success and error messages when necessary. The program will continue to run until the user selects [0] to exit.*
The program was designed to handle queues under the following conditions:

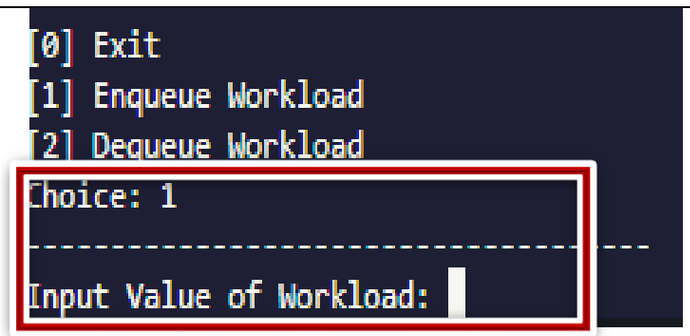
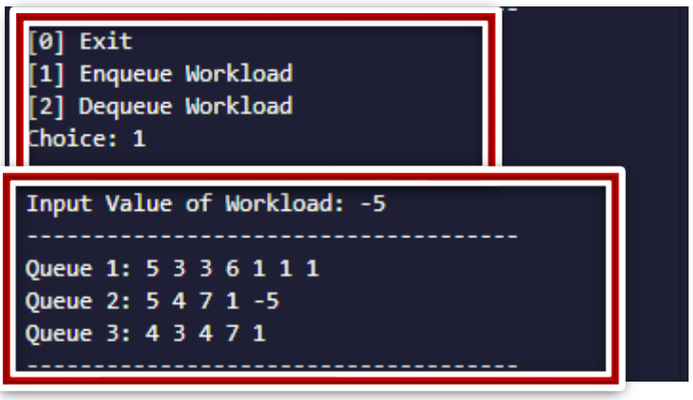
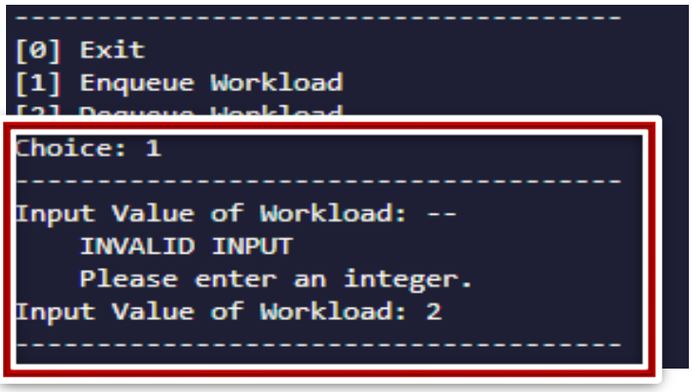
***a**. All queues contain POSITIVE INTEGERS ONLY;*
***b**. All queues contain NEGATIVE INTEGERS ONLY;*
***c**. All queues can be both contain with a value of BOTH POSITIVE and NEGATIVE integers;*

*Additionally, **qload()** (**allows for finding the total queue workload without removing it**), **print_queue()** (**displays the current state of each queues**), **input_enq** (**checks if any queues is empty and check where to load the every added elements**). While **choose_dq()** (**check and choose the queue to be dequeued with the lowest head values**).*

USER GUIDE

Step by step instructions on how to use your program. Include images for easily visualization

Step 1 *The user can run the code in any Python IDE or using a Visual Studio Code.* *After you successfully run the code, the terminal will display this setup of your queue initiating that the queues is empty and thus have no elements/values input to it.*	*Image 1* 
Step 2 *Then below the initiating of empty queues, the user will then be asked the user with three choices. [0] Exit, [1] to enqueue workload, [2] to dequeue workload.* *Entering integers/ numbers other than [0],[1] and [2] will run and prompt an error message and the program will then be asking you again to enter the right choices.*	 *CASE OF AN INVALID INPUT/s:* 

Step 3 *After entering valid integers/numbers to the choices if the user selected to input and prompt '[1] Enqueue Workload'. The terminal will then next to prompt and asked the user to input a value of workload to be added to the queues. The range accepted in inputting value of workload are consist of both negative and positive integers.* *If the user inputted any keys other than integers/numbers may it be positive or negative the terminal will then prompt an error message and the program will ask you to enter the input value of workload again. If the user successfully input the value of workload. The program will then asked you again of the three choice. [0] Exit, [1] to enqueue workload, [2] to dequeue workload.*	 *CASE FOR ACCEPTING NEGATIVE INTEGERS*  *INVALID INPUT FOR VALUE OF WORKLOAD* 
Step 4 *After entering valid integers/numbers to the choices if the user selected*	

to input and prompt '[2] Dequeue Workload'. The terminal will then next to prompt and delete the front with the lowest value of integers that is added before. The range accepted in dequeuing value of workload are consist of both negative and positive integers. The program will then asked you again of the three choice. [0] Exit, [1] to enqueue workload, [2] to dequeue workload.

.

If the user prompt to input and choose again to dequeue workload if and if the queue has only one values the queue will then be all empty. The program will then asked you again of the three choice. [0] Exit, [1] to enqueue workload, [2] to dequeue workload

```
Queue 1: 2
Queue 2: 3
Queue 3: 6
```

```
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 2
-----------------------------------
Queue 1: Empty
Queue 2: 3
Queue 3: 6
```

CASE OF EMPTYING THE QUEUE

```
Queue 1: 2
Queue 2: 3
Queue 3: 6
-----------------------------------
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 2
-----------------------------------
Queue 1: Empty
Queue 2: 3
Queue 3: 6
-----------------------------------
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 2
```

```
Queue 1: Empty
Queue 2: Empty
Queue 3: 6
-----------------------------------
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 2
-----------------------------------
Queue 1: Empty
Queue 2: Empty
Queue 3: Empty
-----------------------------------
```

<table>
<tr>
<td></td>
<td>

```
Queue 1: Empty
Queue 2: Empty
Queue 3: Empty
-------------------------------------
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 2
-------------------------------------
CANNOT DEQUEUE.
All Queues empty.
-------------------------------------
Queue 1: Empty
Queue 2: Empty
Queue 3: Empty
-------------------------------------
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice:
```

CASE OF AN EMPTY QUEUE
</td>
</tr>
<tr>
<td>

Step 5

After entering valid integers/numbers to the choices if the user selects to input and prompt '[0]' Exit. The terminal will then next to prompt and say Thank you!

This will then exit and end the main menu and terminating the program.
</td>
<td>

CASE FOR EXITING THE PROGRAM

```
[0] Exit
[1] Enqueue Workload
[2] Dequeue Workload
Choice: 0
Thank you!
-------------------------------------
```
</td>
</tr>
</table>

PROGRAM CODE

```
print("QUEUE - Linked List Implementation")

class QNode: # for creating Queue node object
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue: # for creating Queue object
    def __init__(self, name):
        self.name = name
        self.head = None

    # to print queue name and contents
    def print_queue(self):
        # in case of an empty queue
        if self.head == None:
            print(f"{self.name}: Empty")

        else: # if queue is not empty
            list = [] # holder for queue elements
            ptr = self.head
            while ptr:
                # adding elements to list
                list.append(str(ptr.data))
                ptr = ptr.next
            print(f"{self.name}: {' '.join(list)}")

    # to find total queue workload
    def qload(self):
        ptr = self.head
        load = 0
        while ptr != None:
            # adding every element value as workload
            load += int(ptr.data)
            ptr = ptr.next
        return load

    # to add element/node at tail
    def enqueue(self, value):
        # in case of an empty queue
        if self.head == None:
            self.head = QNode(value)

        else: # if queue not empty
            ptr = self.head
            # to find last node
            while ptr.next != None:
                ptr = ptr.next
            # adding new node after last node
            ptr.next = QNode(value)
```

```
    # delete element/node at start
    def dequeue(self):
        # in case of an empty queue
        if self.head == None:
            print("Cannot dequeque. Queue is empty.")

        else: # if queue not empty
            ptr = self.head
            self.head = ptr.next # setting new node as head

# creating our 3 queues
q1 = Queue("Queue 1")
q2 = Queue("Queue 2")
q3 = Queue("Queue 3")

# to check input & choose which queue the input will be added to
def input_enq():
    while True:
        uinput = input("Input Value of Workload: ") # input prompt
        try:
            uinput = int(uinput) # testing if input is a valid integer
            break
        except:
            print("    INVALID INPUT \n    Please enter an integer.")

    # comparing queue workloads
    if (q1.qload() <= q2.qload()) and (q1.qload() <= q3.qload()):
        # if Queue 1 has the least workload
        q1.enqueue(uinput)
    elif (q2.qload() < q1.qload()) and (q2.qload() <= q3.qload()):
        # if Queue 2 has the least workload
        q2.enqueue(uinput)
    else: # if Queue 3 has the least workload
        q3.enqueue(uinput)

# to choose which queue to dequeue from
def choose_dq():
    # putting queues in a list for iteration
    queues = [q1, q2, q3]

    # setting a ceiling value for queue head values
    c = float('inf')

    for q in queues:
        # finding the queue with lowest head value
        # enables dequeueing when there are empty queues
        if q.qload() != 0 and q.head.data < c:
            c = q.head.data
            chosen_q = q # queue with lower/st head value
    if c == float('inf'):
        print("-------------------------------------")
        print("CANNOT DEQUEUE. \nAll Queues empty.")
```

```
    else:
        chosen_q.dequeue()

# menu/interface function
def program():
    while True:

        print("-------------------------------------")
        q1.print_queue()
        q2.print_queue() # displaying queues at every loop of menu
        q3.print_queue()
        print("-------------------------------------")

        print("[0] Exit \n[1] Enqueue Workload \n[2] Dequeue Workload")

        while True:
            choice = input("Choice: ")

            # E X I T
            if choice == "0":
                print("Thank you!")
                print("-------------------------------------\n")
                return

            # E N Q U E U E
            elif choice == "1":
                print("-------------------------------------")
                input_enq()
                break

            # D E Q U E U E
            elif choice == "2":
                choose_dq()
                break

            # if user input is not 0, 1, or 2
            else:
                print("    INVALID INPUT")

program() # calls program function/runs the menu
```

TUTORIAL VIDEO

YouTube Link: https://www.youtube.com/watch?v=_ORGyol8LsI

TAKEAWAYS

Name of Member: **LANUZO,** JESSICA MAE T.
Learnings: *To always consider the case where the list, stack, queue, or object is empty. Comments can be helpful for understanding your own code, especially if you haven't looked at it since you wrote it. School tasks shouldn't take a long time; sometimes we struggle with making decisions or figuring out how to approach them. Also we're not early in finishing this learning task; we should have started it sooner to make room for tasks from other courses. Lastly, prioritizing tasks with the least workload is one of the 'approaches' our OS instructor mentioned, involving FIFO, LIFO, and the prioritization we implemented in this queue LT.*

Name of Member: **COLICO**, CARL RENZ M.
Learnings: *I discovered that implementing a Queue LDS using a linked list in Python is quite straightforward. It involves utilizing basic built-in Python methods such as append(), int, and while/else loops. Understanding how these operations streamline the coding process and improve the functionality of queues has helped me visualize their real-world applications. During the test run, we encountered some minor errors. One issue was the lack of structured code for catching empty queues when the user tried to dequeue. Instead of handling the situation properly, the program ran into a terminal error, causing the program to halt and hinder the processes. Fortunately, one of our team members quickly resolved the issue. From that I learned what and how is the behaviour of queues than other LDS structure, its logic programs and its content variety in handling the inputted elements/integers in the list of queues. I did also learned that doing the tasked early will give you more chances of fixing mistakes like in our code, also it did boost my productivity in doing also other task and also it did give me more time in carving and processing on how the overall structure of code connected to each other and how each of its define methods is depict while I run and test it in our terminal and also the most important is it reduces to stress and feel overwhelm how time is shorten bit by bit as deadline approaches.*

Name of Member: **TERCERO,** MICHELLE S.
Learnings: *The implementation of Queue LDS is quite different from other types of LDS. Queues utilize a FIFO approach, making their logical behavior different from other LDS. I also learned and applied how the methods used in this learning task apply in the real world. I also learned how the queue works, such as enqueue and dequeue of integers or elements being appended or deleted in the queue, and how they are positioned to be added or deleted in some way. I actually enjoy demonstrating the output but also*

struggle to understand the codes sometimes but with the help of the comments it made me understand it. This activity helped me understand more how the FIFO and LIFO works. This experience made me realize that I need to dedicate more time to studying this topic in depth and avoid the habit of cramming my schoolwork. I understand now that a more consistent and thorough approach to learning will help me grasp the material better and reduce stress before deadlines. Lastly, this learning task out of all the three is one of the tasked we did submit on rush as we did not finished it early.